

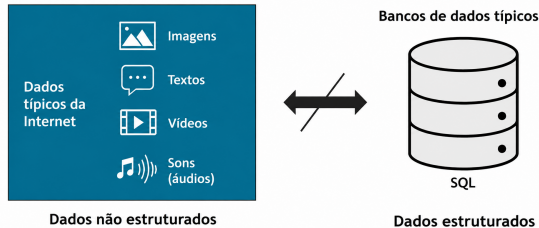
Unidade 4 — Banco de Dados Vetoriais

Aplicações de Modelos de Linguagem Natural

- Vimos que o funcionamento do RAG depende diretamente de uma **base de dados separada**, capaz de armazenar conhecimento externo e de ser consultada de forma eficiente pelo recuperador.
- Essa base é o **banco de dados vetorial**.
- Esta seção é dividida em cinco partes:
 - ❶ **3.1** O que é um banco de dados vetorial;
 - ❷ **3.2** Como os documentos são divididos em blocos antes de entrar nele (*chunking*);
 - ❸ **3.3** Como esses blocos viram vetores (*embeddings*);
 - ❹ **3.4** Como esses dados são armazenados;
 - ❺ **3.5** Como eles são recuperados quando uma consulta chega.

3.1 O que é um Banco de Dados Vetorial

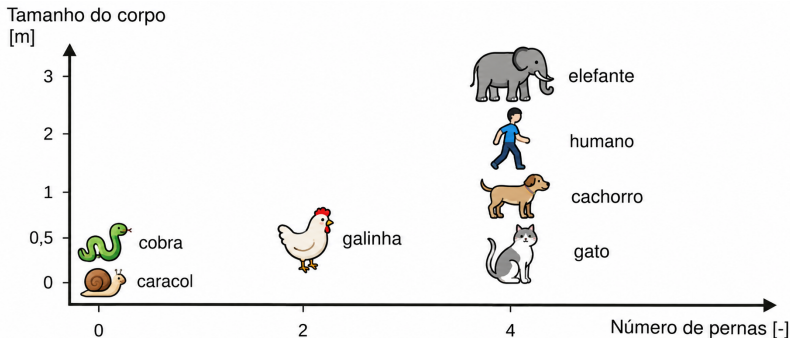
- Bancos de dados tradicionais são capazes de armazenar e recuperar **dados estruturados**, como tabelas com colunas e linhas bem definidas.
- Mas isso não reflete a grande variedade de formatos do dia a dia: além de texto, também há imagens, arquivos de áudio e vídeos.
- Todos esses formatos precisam ser tratados de forma parecida e armazenados em um mesmo tipo de banco de dados — algo que os bancos tradicionais não fazem bem.
- Por isso surgiu um novo paradigma: o **banco de dados vetorial**.



3.1 Por que “vetorial”?

- O banco é chamado de “vetorial” porque **todos os tipos de dado podem ser representados como vetores**.
- Relembrando a matemática básica: um vetor é uma linha com uma seta, desenhada em um sistema de coordenadas, que pode ser representada por um conjunto de números.
- Exemplo: imagine um gráfico bidimensional que representa seres vivos segundo duas características — o **número de pernas** e o **tamanho do corpo**.
- Representando a informação dessa forma, é possível perceber o significado semântico dos conceitos: gatos e cachorros, por exemplo, ficam bem próximos, porque têm características parecidas nessas duas dimensões.

3.1 Um espaço vetorial bidimensional



Conceitos mapeados para vetores (número de pernas, tamanho do corpo)

Cachorro $\rightarrow [4, 1]$

Ser humano $\rightarrow [4, 2]$

Cobra $\rightarrow [0, 0.5]$

Gato $\rightarrow [4, 0.5]$

Elefante $\rightarrow [4, 3]$

Caracol $\rightarrow [0, 0.1]$

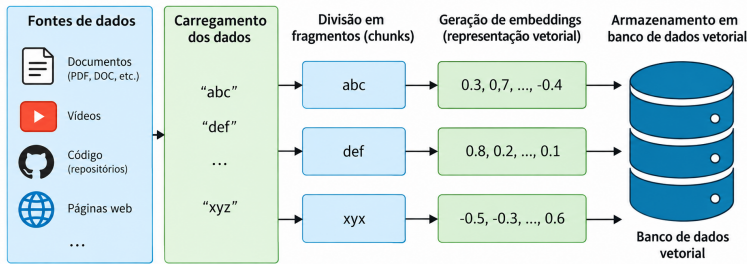
3.1 Espaços de alta dimensão e vetores de embedding

- Como seres humanos, conseguimos imaginar um ponto em um espaço bidimensional ou até tridimensional, mas não conseguimos imaginar um espaço de **768 ou 3.072 dimensões** — justamente o tamanho típico dos vetores gerados pelos modelos de embeddings atuais.
- Computadores, sim, conseguem lidar com esses espaços de altíssima dimensão, o que permite representar o significado semântico de palavras, imagens, sons ou qualquer outro tipo de informação de forma muito mais rica.
- O aspecto fundamental é que **conceitos semanticamente parecidos ficam posicionados mais próximos entre si do que conceitos muito diferentes**.
- Quanto mais dimensões um algoritmo tem à disposição, melhor tende a ser sua compreensão do significado de uma palavra ou texto.
- O processo de traduzir textos legíveis por humanos em vetores é chamado de **embedding**, uma das etapas da ingestão de dados em um banco de dados vetorial.

3.1 As quatro etapas da ingestão de dados

Sempre que se deseja adicionar dados a um banco de dados vetorial, passa-se por quatro etapas:

- 1 **Carregamento dos dados**, a partir de uma fonte, como arquivos de texto, páginas web ou outros documentos;
- 2 **Divisão em blocos menores** (*chunking*);
- 3 **Criação dos embeddings**: tradução de cada bloco de texto em um vetor numérico;
- 4 **Armazenamento** desses pares texto/vetor no banco de dados.



3.2 Chunking — Por que dividir os documentos?

- Frequentemente é necessário dividir um documento longo em blocos menores (*chunks*) antes de armazená-lo em um banco de dados vetorial.
- Essa divisão é importante por três motivos principais:
 - 1 Blocos menores tendem a conter **informação mais focada e relevante**, o que melhora a granularidade dos resultados de busca.
 - 2 A divisão contorna **limitações inerentes aos modelos de embeddings**, que só conseguem processar uma quantidade limitada de texto por vez.
 - 3 Trabalhar e armazenar blocos menores tende a ser **mais eficiente** em termos de tamanho do banco de dados e de velocidade das consultas.

3.2 Chunking — Tokens e tokenização

- Para entender o segundo motivo, é importante conhecer o conceito de **token**: a unidade fundamental de texto em processamento de linguagem natural, normalmente uma palavra inteira, parte de uma palavra ou até um único caractere.
- O processo de converter uma frase em uma lista de tokens é chamado de **tokenização**.

Exemplos de tokenização

“Eu amo IA generativa!” → [“Eu”, “amo”, “IA”, “generativa”, “!”]

Em muitos métodos, palavras mais longas são quebradas em pedaços menores: “amando” → [“am”, “ando”]

- Os tokens permitem que os modelos de IA processem texto convertendo linguagem em representações numéricas; em seguida, na etapa de embedding, esses tokens são transformados em vetores que capturam o significado semântico.
- Como regra geral, **1 token $\approx$ 0,75 palavra** (ou seja, 100 tokens correspondem a cerca de 75 palavras, em inglês).

3.2 Chunking — Janela de contexto

- Todo modelo de embeddings tem um limite máximo de tokens que consegue processar de uma vez. Esse limite é chamado de **janela de contexto** (*context window*).
- Se mais tokens do que esse limite forem enviados, o excesso é simplesmente **descartado**, perdendo-se o significado dessa parte do texto.
- Por isso os documentos precisam ser divididos em blocos que caibam na janela de contexto do modelo de embeddings.

Janelas de contexto de modelos de embeddings populares

OpenAI text-embedding-3-small: 8.191 tokens

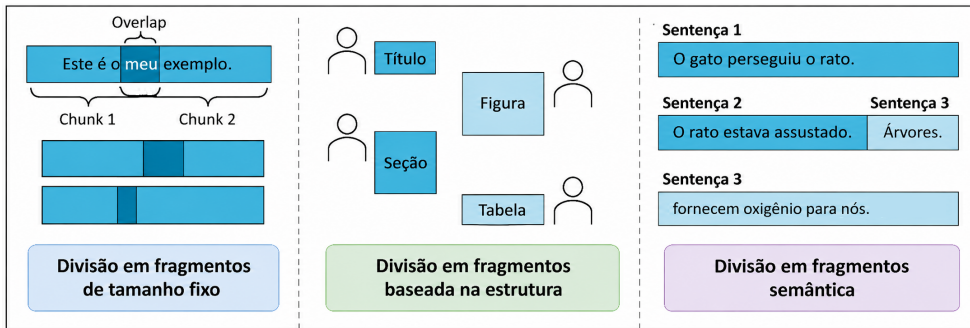
OpenAI text-embedding-3-large: 8.191 tokens

Anthropic voyage-3-large: 32.000 tokens

All-MiniLM-L6-v2: 256 tokens

3.2 Chunking — Três estratégias de divisão

Existem diferentes abordagens para criar os blocos de texto (*chunks*). A seguir, apresentamos três das mais utilizadas (Figura 6).



3.2 Chunking — Divisão de tamanho fixo

- **Divisão de tamanho fixo (*fixed-size chunking*):** a abordagem mais simples. O texto é dividido sempre que um número predefinido de caracteres é atingido, com o corte no final da última palavra completa antes de ultrapassar esse limite.
 - **Vantagem:** simplicidade — é fácil de entender, de implementar e é rápida, já que não exige nenhuma análise do significado do texto.
 - **Desvantagem:** falta de sensibilidade ao contexto — frases ou ideias podem ser cortadas no meio, resultando em perda de coerência semântica.
- **Sobreposição entre blocos (*chunk overlap*):** recurso importante nessa e em outras abordagens. É a quantidade de texto compartilhada entre dois blocos consecutivos, normalmente medida em caracteres ou tokens.
 - O objetivo é manter o contexto entre os blocos e preservar o significado que poderia se perder quando o texto é dividido de forma arbitrária.
 - Uma regra geral comum é usar uma sobreposição de **10% a 20% do tamanho do bloco** — mas não é uma regra fixa: dependendo do tipo de texto, mais ou menos sobreposição pode ser adequada.

3.2 Chunking — Divisão baseada na estrutura do documento

- O significado e a coerência de um texto normalmente não seguem tamanhos fixos; dividir o texto em pedaços de tamanho igual nem sempre reflete bem a realidade.
- **Divisão baseada na estrutura do documento (*structure-based chunking*)**: uma abordagem mais sofisticada, que segue a própria estrutura do documento, organizado pelo autor em capítulos, parágrafos, frases, partes de frases, palavras, e assim por diante.
- A ideia é quebrar primeiro o texto em unidades maiores, como capítulos; se o bloco resultante ainda for grande demais, ele é quebrado em unidades menores, como parágrafos, e assim sucessivamente, até respeitar o limite de tamanho definido.
- Essa abordagem equilibra bem a **coerência textual** com o cumprimento dos **limites de tamanho** dos blocos.

Exemplo: histórico de conversa

Se os blocos de texto de duas pessoas diferentes ficassem misturados por causa de uma divisão mal feita, a compreensão da conversa ficaria prejudicada — como ler um livro em que as falas fossem atribuídas às pessoas erradas.

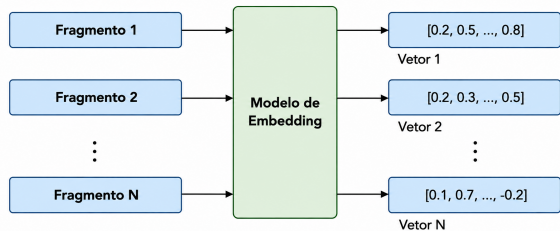
3.2 Chunking — Divisão semântica

Divisão semântica (*semantic chunking*): a abordagem mais avançada das três.

- ❶ O texto é dividido em frases individuais.
 - ❷ Cada frase é transformada em um vetor de embedding.
 - ❸ Calcula-se a similaridade entre frases consecutivas (por meio de uma espécie de matriz de distâncias).
 - ❹ A divisão em blocos é feita nos pontos em que essa similaridade cai de forma expressiva — as “**quebras naturais**” de assunto dentro do texto.
- **Vantagens:** preserva melhor o significado semântico das frases e mantém conceitos relacionados no mesmo bloco, o que melhora a qualidade da recuperação; além disso, adapta-se ao conteúdo, produzindo blocos de tamanhos mais naturais e mais fáceis de entender.
 - **Desvantagens:** exige processamento computacional adicional (é mais lenta), pois depende de rodar um modelo de embeddings sobre o texto — o que tem um custo em tempo de resposta (latência), financeiro ou em complexidade adicional do sistema.

3.3 Embeddings

- Depois de carregar e dividir os documentos em blocos, ainda estamos lidando apenas com texto legível por humanos.
- É preciso traduzir esse texto para uma representação compreensível por máquina: o **embedding**.
- Nesse processo, cada bloco de texto (*chunk*) é convertido em um vetor numérico.



3.3 Como escolher um modelo de embeddings

Ao escolher um modelo de embeddings, alguns fatores importantes devem ser considerados:

- ❶ **Desempenho do modelo:** existem rankings públicos, como o *Massive Text Embedding Benchmark* (MTEB), que comparam dezenas de modelos em diferentes tarefas e idiomas.
- ❷ **Tamanho do modelo e uso de memória:** importantes caso a intenção seja hospedar o próprio modelo.
- ❸ **Tamanho máximo de entrada:** é preciso garantir que o tamanho dos blocos de texto fique dentro desse limite.
- ❹ **Custo:** muitos modelos são oferecidos como serviço pago via API, enquanto outros são de código aberto e gratuitos — embora hospedá-los localmente exija manter um servidor (com custos de hardware e energia).

Privacidade

A vantagem de hospedar o próprio modelo é evitar que os dados da organização saiam de sua própria rede, o que pode ser importante em cenários com restrições de privacidade.

3.4 Armazenamento dos Dados

- Depois de carregar os documentos, dividi-los em blocos e gerar os embeddings, chega-se à última etapa da ingestão: **armazenar os pares (texto e vetor)** em um banco de dados vetorial.
- A principal decisão é se o banco será **hospedado localmente** (na própria infraestrutura da organização) ou se será usado um **serviço de nuvem** gerenciado por terceiros. Cada abordagem tem vantagens e desvantagens.

Critérios importantes para escolher um banco de dados vetorial:

Localização

Alguns bancos armazenam os dados localmente (por exemplo, o **Chroma**, opção de código aberto que pode rodar na própria máquina); outros são serviços de nuvem (por exemplo, **Pinecone** ou **Weaviate**).

Preço

Bancos locais e de código aberto são gratuitos, mas exigem manter um servidor próprio, com hardware adequado para gerar embeddings rapidamente: trocam-se custos variáveis (serviço de nuvem) por custos fixos mais altos (infraestrutura própria).

Proteção de dados

Dependendo do tipo de dado e do cliente, os dados podem ser altamente sensíveis, levando à necessidade de evitar serviços online ou determinadas regiões geográficas. Nesses casos, um banco local pode ser preferível.

Desempenho versus velocidade

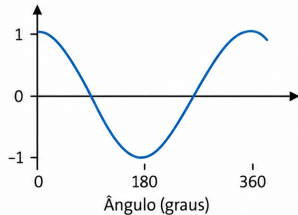
Em geral, melhor desempenho (qualidade dos resultados) está correlacionado a espaços de embedding maiores, o que torna mais lentas tanto a inserção/atualização de dados quanto as consultas.

3.5 Recuperação de Dados — Similaridade de cosseno

- Diferentemente dos bancos estruturados tradicionais, que sempre buscam **correspondências exatas**, um banco de dados vetorial busca os documentos **mais relevantes** para uma consulta, não necessariamente uma correspondência perfeita.
- Duas abordagens de similaridade são especialmente importantes: a **similaridade de cosseno** e a **Relevância de Margem Máxima** (*Maximum Margin Relevance*, MMR).
- **Similaridade de cosseno**: o valor do cosseno é máximo quando o ângulo entre dois vetores está próximo de zero. Vetores que apontam praticamente na mesma direção são considerados similares; se apontam em direções opostas, são o mais diferentes possível. O valor sempre fica entre -1 e $+1$.
- Embora o espaço de embeddings tenha muito mais do que duas dimensões, o mesmo princípio se aplica. A recuperação segue três passos:
 - 1 calcular o ângulo (e o cosseno correspondente) entre o vetor da consulta e o vetor de cada documento (ou bloco de texto) armazenado;
 - 2 ordenar os documentos em ordem decrescente de similaridade;
 - 3 retornar os “**top k**” resultados — por exemplo, apenas os 2 ou os 5 documentos mais similares à consulta.

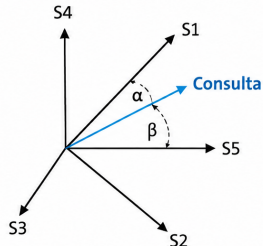
3.5 O princípio da similaridade de cosseno

Função cosseno

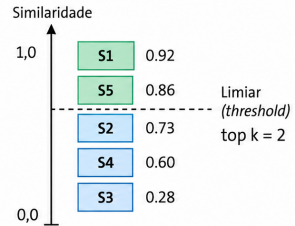


Valor máximo para o menor ângulo

Representação vetorial simplificada



Similaridade do cosseno



Recuperação (documentos mais similares)

3.5 Relevância de Margem Máxima (MMR)

- A similaridade de cosseno ajuda a encontrar os documentos mais relevantes, mas em alguns casos isso não é suficiente.
- É aí que entra a **Relevância de Margem Máxima (MMR)**: uma técnica que busca maximizar não apenas a **relevância**, mas também a **diversidade** dos documentos retornados.

Consulta: “Quais são as principais atrações turísticas de Nova York?”

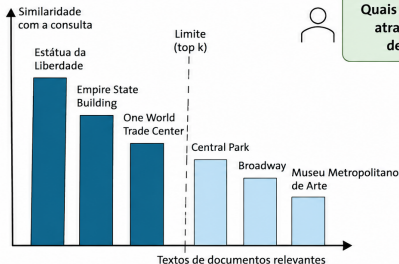
Apenas similaridade de cosseno: três arranha-céus muito parecidos entre si (por exemplo, dois deles são prédios muito altos e semelhantes).

Usando MMR: um marco histórico, um parque e um distrito cultural — resultados relevantes e, ao mesmo tempo, mais diversos entre si.

3.5 O hiperparâmetro λ da MMR

- O funcionamento da MMR depende de um hiperparâmetro chamado λ (**lambda**), que varia entre 0 e 1:
 - $\lambda = 0$: retorna apenas os resultados mais relevantes (equivalente à similaridade de cosseno pura);
 - $\lambda = 1$: retorna os resultados mais diversos possíveis, mesmo que um pouco menos relevantes individualmente;
 - $\lambda = 0,5$: costuma ser um bom ponto de partida, equilibrando relevância e diversidade.

a) Similaridade do cosseno



Quais são as principais atrações turísticas de Nova York?

b) MMR (Diversidade de resultados)

